Programming Assignment 1

Sanoj S Vijendra
22b0916

Table of Content

1. **Task 1: Algorithm Descriptions**
    a. Algorithm 1 (UCB)
        i. `init` function
        ii. `give_pull` function
        iii. `get_reward` function
    b. Algorithm 2 (UCB-KL)
        i. `give_pull` function
        ii. `get_reward` function
    c. Algorithm 3 (Thompson Sampling)
        i. `init` function
        ii. `give_pull` function
        iii. `get_reward` function
2. **Task 2: Breaking the Door**
    a. `init` function
    b. `select_arm` function
    c. `get_reward` function
    d. Explanation
3. **Task 3: KL-UCB vs Optimised KL-UCB**
    a. Optimization Approach
    b. Code
        i. `init` function
        ii. `kl_div` function
        iii. `get_ucb` function
        iv. `give_pull` function
            1. Initial Exploration Phase and Batch Continuation
            2. New Batch Selection
            3. Batch Size Determination
        v. `get_reward`
    c. Computational Complexity
    d. Performance Analysis
        i. Regret Optimality
        ii. Computational Speedup
        iii. Plots

Task 1:

1. **Algorithm 1 (UCB):**

```
class UCB(Algorithm):
    def __init__(self, num_arms, horizon):
        super().__init__(num_arms, horizon)
        # You can add any other variables you need here
        # START EDITING HERE
        self.values = np.zeros(num_arms)
        self.counts = np.zeros(num_arms)
        self.total_counts = 0
        self.c = 2  # exploration param
        # END EDITING HERE

    def give_pull(self):
        # START EDITING HERE
        self.total_counts+=1
        for a in range(self.num_arms):
            if self.counts[a] == 0:
                return a
        ucb_values =
self.values+np.sqrt((self.c*np.log(self.total_counts))/(self.counts)
)
        return int(np.argmax(ucb_values))
        # END EDITING HERE


    def get_reward(self, arm_index, reward):
        # START EDITING HERE
        self.counts[arm_index]+=1
        n = self.counts[arm_index]
        value = self.values[arm_index]
        new_value = ((n - 1) / n) * value + (1 / n) * reward
        self.values[arm_index] = new_value
        # END EDITING HERE
```

a) init function:

- “values”, “counts”, “total_counts” and “c” has been initialised.
- values: Contains list of empirical means for arm i at index i.
- counts: Contains list of number of pull for each arm i at index i.
- total_counts: Total number of pulls.
- c: Constant parameter for ucb.

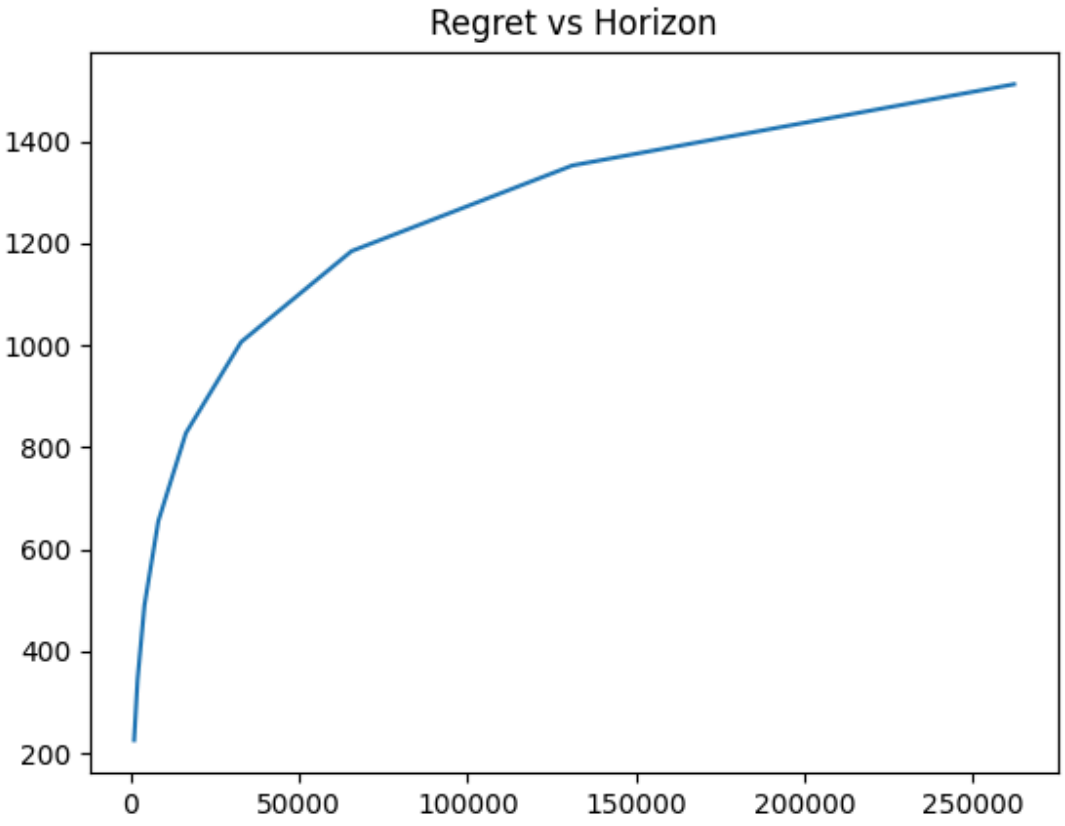
b) give_pull function:

- Initially each arm is pulled at-least once.
- For each arm sum (ucb) of empirical means and exploration bonus is calculated.
- The arm with maximum ucb is chosen. In case of tie minimum index i is chosen.

c) get_reward function:

- counts for corresponding arm is incremented.
- A new empirical mean is calculated for that arm and the empirical mean is updated.

Plot:



Regret (on y) **vs Horizon** (on x**) for UCB**

2. Algorithm 2 (UCB-KL):

```
def kl_div(p, q):
    p = min(max(p, 1e-15),1- 1e-15)
    q = min(max(q, 1e-15),1- 1e-15)
    return p*math.log(p/q)+(1-p)*math.log((1-p)/(1-q))
class KL_UCB(Algorithm):
    def __init__(self, num_arms, horizon):
        super().__init__(num_arms, horizon)
        # START EDITING HERE
        self.values = np.zeros(num_arms)
        self.counts = np.zeros(num_arms)
        self.total_counts = 0
        self.c = 3  # exploration param
        # END EDITING HERE
    def give_pull(self):
        # START EDITING HERE
        self.total_counts+=1
        for a in range(self.num_arms):
            if self.counts[a] == 0:
                return a
        kl_ucb_values = np.zeros(self.num_arms)
        for a in range(self.num_arms):
            low, high = self.values[a],1.0
            while(high-low>1e-6):
                mid=(low+high)/2.0

if(kl_div(self.values[a],mid)>(math.log(self.total_counts)+self.c*ma
th.log(max(math.log(max(self.total_counts,2)),1.0000000001)))/self.c
ounts[a]):
                    high = mid
                else:
                    low = mid
            kl_ucb_values[a] = (high+low)/2.0
        return int(np.argmax(kl_ucb_values))
        # END EDITING HERE
    def get_reward(self, arm_index, reward):
        # START EDITING HERE
        self.counts[arm_index]+=1
        n = self.counts[arm_index]
```

```
        value = self.values[arm_index]
        new_value = ((n - 1) / n) * value + (1 / n) * reward
        self.values[arm_index] = new_value
```

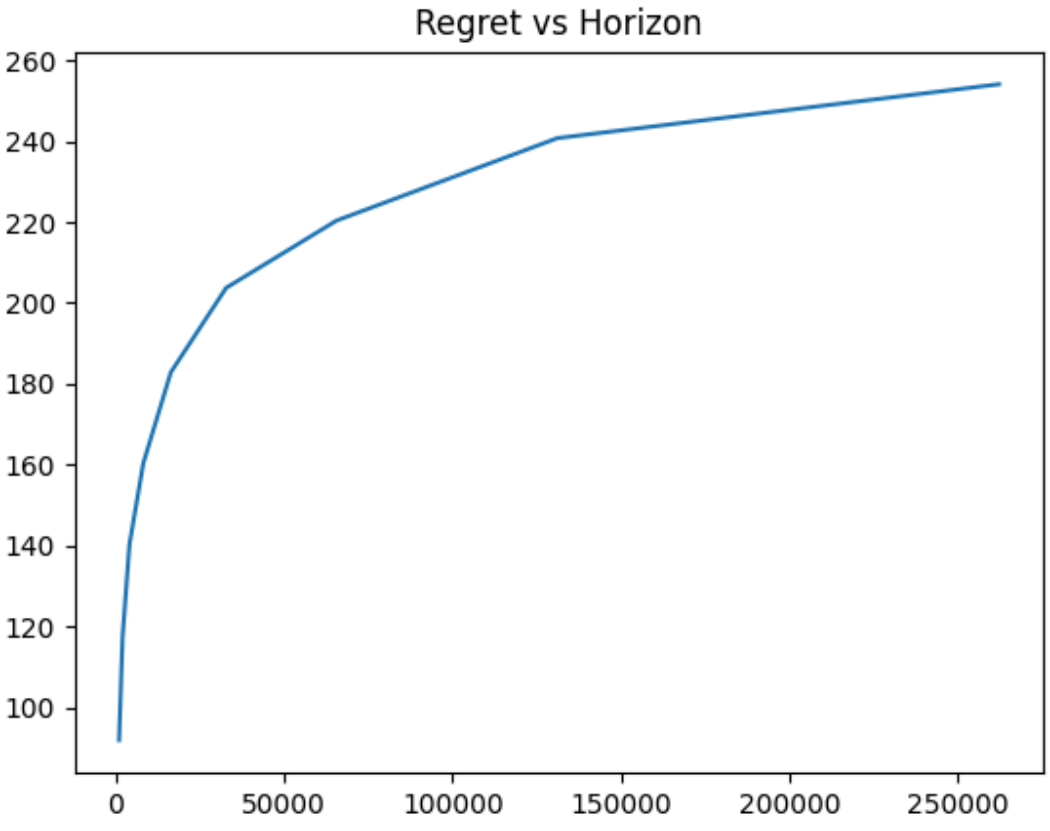
a) give_pull function:

- Initially each arm is pulled at-least once.
- For each arm, ucb-kl value is calculated. Calculation is made using binary search algorithm. (Binary search is applicable as kl divergence is an increasing function for given p)
- The arm with maximum ucb-kl is chosen. In case of tie minimum index i is chosen.

b) get_reward function:

- counts for the corresponding arm is incremented.
- A new empirical mean is calculated for that arm and the empirical mean is updated.

Plot:



Regret (on y) **vs Horizon** (on x**) for UCB-KL**

3. **Algorithm 3 (Thompson Sampling):**

```
class Thompson_Sampling(Algorithm):
    def __init__(self, num_arms, horizon):
        super().__init__(num_arms, horizon)
        # You can add any other variables you need here
        # START EDITING HERE
        self.success = np.zeros(num_arms)
        self.failure = np.zeros(num_arms)
        self.total_counts = 0
        # END EDITING HERE

    def give_pull(self):
        # START EDITING HERE
        # Sample from Beta distribution for each arm
        self.total_counts+=1
        x = [np.random.beta((self.success[a]+1),(self.failure[a]+1))
for a in range(self.num_arms)]
        return int(np.argmax(x))
        # END EDITING HERE

    def get_reward(self, arm_index, reward):
        # START EDITING HERE
        if reward==1:
            self.success[arm_index]+=1
        else:
            self.failure[arm_index]+=1
        # END EDITING HERE
```

a) init function:

- “success”, "failure" and “total_counts” are initialised.
- success: Represents number of times arm i succeeded (reward 1), at index i.
- failure: Represents number of times arm i has failed (reward 0), at index i.
- total_counts: Total number of pulls.

b) give_pull function:

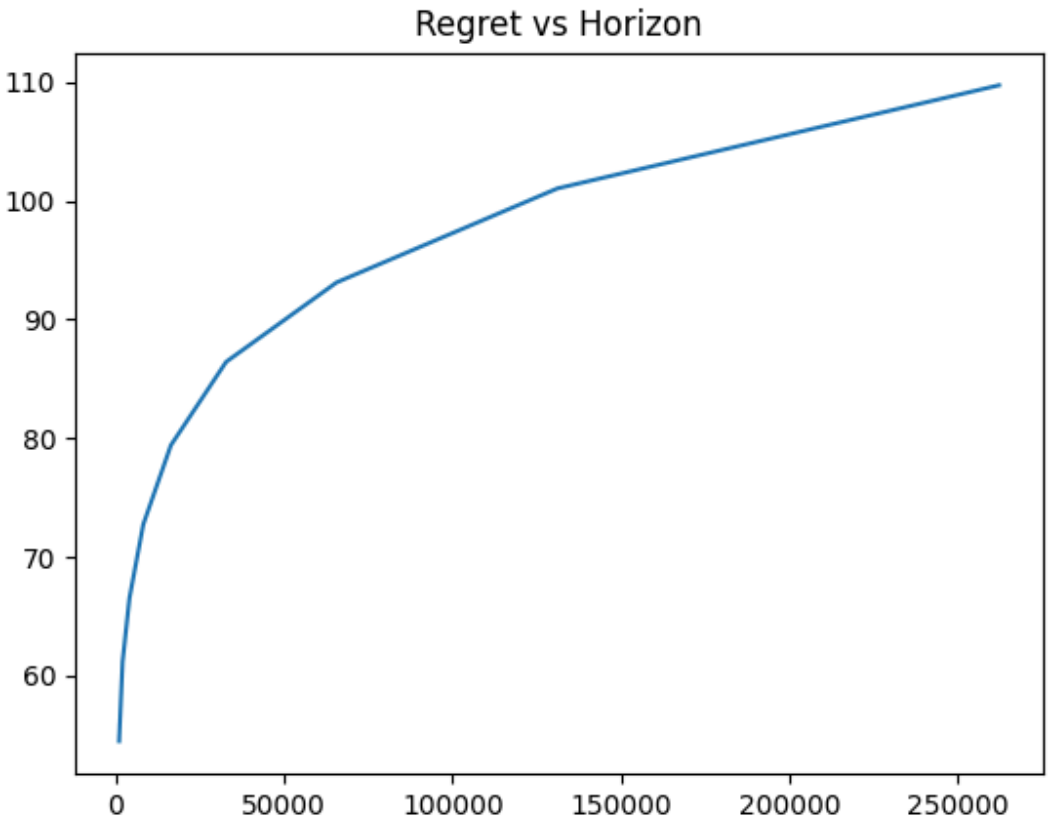
- A random sample is drawn from $\beta(success + 1, faliure + 1)$ for each arm.

- Total pull count is incremented.
- The arm with maximum value for the drawn sample is chosen.chosen.

c) get_reward function:

- If reward is 1, success for that arm is incremented.
- Otherwise, failure for that arm is incremented.

Plot:



Regret (on y) **vs Horizon** (on x**) for Thompson Sampling**

Task 2:

Breaking the Door

```
class StudentPolicy(Policy):
    """
    Implement your own algorithm here.
    Replace select_arm with your strategy.
    Currently it has a simple implementation of the epsilon greedy
strategy.
    Change this to implement your algorithm for the problem.
    """
    def __init__(self, K: int, rng: Optional[np.random.Generator] = None):
        super().__init__(K, rng)
        self.health = np.full(K,100.0)

    def select_arm(self, t: int) -> int:
        ind = np.arange(self.K)
        np.random.shuffle(ind)
        for a in range(min(self.K,10)):
            if self.counts[a] == 0:
                return a
        mus_hat = self.means.copy()
        eps=1e-15
        mus_hat = np.where(mus_hat>0, mus_hat, eps)
        scores = self.health/mus_hat
        return int(np.argmin(scores))

    def reset_stats(self):
        super().reset_stats()
        self.health = np.full(self.K,100.0)

    def update(self, arm: int, reward: float):
        super().update(arm,reward)
        self.health[arm]-=reward
```

a) init function:
    - Apart from superclass init function, a health list is declared here, which will contain the current health of each door i, at index i.

b) select_arm function:
- A list of all the numbers from 0 to K-1 is created.
- Then a random permutation of the above list is created.
- Among the first 10 arms present in the permutation (for K>10, K for K<=10), if any arm is not pulled even a single time then that arm is pulled. (reasoning in explanation)
- Else, expected number of hits required to break each door is calculated based on present empirical means and present health of each door.
- The arm with the lowest expected hit is chosen.

c) get_reward function:
- Count and total reward are updated in the super class function. Along with those, the health of the corresponding door is decremented by the reward.

Explanation:

Our objective for this task is to design a strategy that breaks any one door among all, in minimum expected number of strikes.
As the game ends as soon as any door reaches health 0, this means that the game does not go on forever and hence we are not required to optimise the regret in the long run but we need to devise a strategy that finishes the game as soon as possible.
Therefore, we must move greedily here. At first we must hit each door once to start. But as the number of doors increases, exploring each door will also increase our number of hits. So, to counter that we'll shuffle all the indices and create a random permutation. We'll take the first min(10,K) (value of 10 was chosen arbitrarily as it was giving good results on given test cases) indices from them and check if those arms are pulled till now or not. If there is an arm which is not pulled at all we'll pull that arm. By this we keep our exploration in check.
Our greedy approach involves calculating the expected number of hits required to break each door, calculated using current health of the door and empirical mean seen so far. The door with least expected hit will be selected.
Our strategy is practical too. As we do not know at all which door is actually weakest (highest actual mean) so must rely on the empirical data to move forward as well as exploring a few times.

Task 3:

KL-UCB vs Optimised KL-UCB

1. Optimization Approach

The optimization implements a **batched approach** that identifies when the same arm should be pulled multiple times consecutively, eliminating redundant calculations. This is based on the behaviour that in typical KL-UCB runs, there are long sequences where the same arm gets pulled before switching.

2. Code

2.1 init function

```
def __init__(self, num_arms, horizon):
        super().__init__(num_arms, horizon)
        # can initialize member variables here
        #START EDITING HERE
        self.t = 0
        self.counts = np.zeros(num_arms,dtype=int)
        self.sums = np.zeros(num_arms)
        self.c=1.2
        self.curr_arm = 0
        self.rem_pulls = 0
        self.epsilon=1e-15
        #END EDITING HERE
```

2.2 *kl*_div function

```
def kl_div(self, p,q):
        p = max(self.epsilon,min(1-self.epsilon,p))
        q = max(self.epsilon,min(1-self.epsilon,q))
        if abs(p-q)<self.epsilon:
            return 0
        return p*math.log(p/q)+(1-p)*math.log((1-p)/(1-q))
```

This implementation ensures removing unnecessary values by clamping values away from 0 and 1, preventing log(0).

2.3 get_ucb function

```
def get_ucb(self,p, n):
        rhs = math.log(self.t) if self.t > 0 else 0
        if((self.t>1) and (self.c>0)):
            rhs+=self.c*math.log(math.log(self.t))
        target = rhs/n
        l,r=p,1.0
        if abs(p-1.0)<self.epsilon:
            return 1.0
        for _ in range(30):
            q_mid=(l+r)/2.0
            if self.kl_div(p,q_mid)<target:
                l=q_mid
            else:
                r=q_mid
        return (l+r)/2.0
```

The UCB calculation uses binary search to solve the KL-UCB equation efficiently. The number of iterations is fixed to 30 to remove large iterations.

2.4 give_pull function

This is the heart of the optimization and implementation of the batched selection strategy:

2.4.1 Initial Exploration Phase and Batch Continuation

```
if self.t<self.num_arms:
            return self.t
        if self.rem_pulls>0:
            self.rem_pulls-=1
            return self.curr_arm
```

Pulling each arm at least once and continuing the current batch if the number of pulls is greater than 0 in the current batch.

2.4.3 New Batch Selection

```
ucbs = np.zeros(self.num_arms)
       for arm in range(self.num_arms):
           if self.counts[arm]==0:
               ucbs[arm]=float('inf')
               continue
           p = self.sums[arm]/self.counts[arm]
           ucbs[arm] = self.get_ucb(p, self.counts[arm])
       best_arm=np.argmax(ucbs)
       self.curr_arm= best_arm
       ucbs_copy = np.copy(ucbs)
       ucbs_copy[best_arm] = -np.inf
       sec_best = np.argmax(ucbs_copy)
       sec_ucb = ucbs_copy[sec_best]
```

The algorithm computes UCBs for all arms and identifies the best and second-best arms.

2.4.4 Batch Size Determination

When the best arm's mean is below the second-best UCB:

```
if p_best<sec_ucb:
           kl = self.kl_div(p_best, sec_ucb)
           if kl>self.epsilon:
               rhs = math.log(self.t) if self.t > 0 else 0
               if((self.t > 1)and(self.c > 0)):
                   rhs+=self.c*math.log(math.log(self.t))
               N = rhs/kl
               m = math.floor(N)-self.counts[self.curr_arm]
               m = max(1,int(m))
       else:
           p_sec = self.sums[sec_best]/self.counts[sec_best]
           N_sec = self.counts[sec_best]
           ucb_best = ucbs[self.curr_arm]
           if (ucb_best<(1.0-self.epsilon)) and N_sec>0:
               kl_div_race = self.kl_div(p_sec,ucb_best)
               if kl_div_race>self.epsilon:
                   if N_sec*kl_div_race > 50:       # arbitrary value
                        m = self.horizon-self.t
                   else:
                       t_new = math.exp(N_sec*kl_div_race)
```

```
                    m = math.floor(t_new-self.t)
                    m = max(1, int(m))
        self.rem_pulls=min(m,self.horizon-self.t)
        self.rem_pulls-=1
        return self.curr_arm
```

Calculating m, the number of batched pulls for the best arm. When the best arm's lead is uncertain (p_best, is below the second-best UCB), we calculate the KL divergence between these two values. This measures the information gap, telling us the number of pulls m required to confirm its superiority. When the best arm is already ahead, we calculate the KL divergence between the second-best mean and the best UCB to project how long the second best might take to become close to first.

2.4.5 reward function

```
def get_reward(self, arm_index, reward):
        #START EDITING HERE
        self.t += 1
        self.counts[arm_index] += 1
        self.sums[arm_index] += reward
        #END EDITING HERE
```

Standard reward function.

3. Computational Complexity

Standard KL-UCB: O(nTlog(T))

- n arms × T time steps × log T binary search iterations

Optimized KL-UCB: O(n log(T) + Blog(T))

- n arms × log T for batch selection
- B batches × log T for UCB calculations
- Where B << T (typically B = O(log T))

4. Performance Analysis

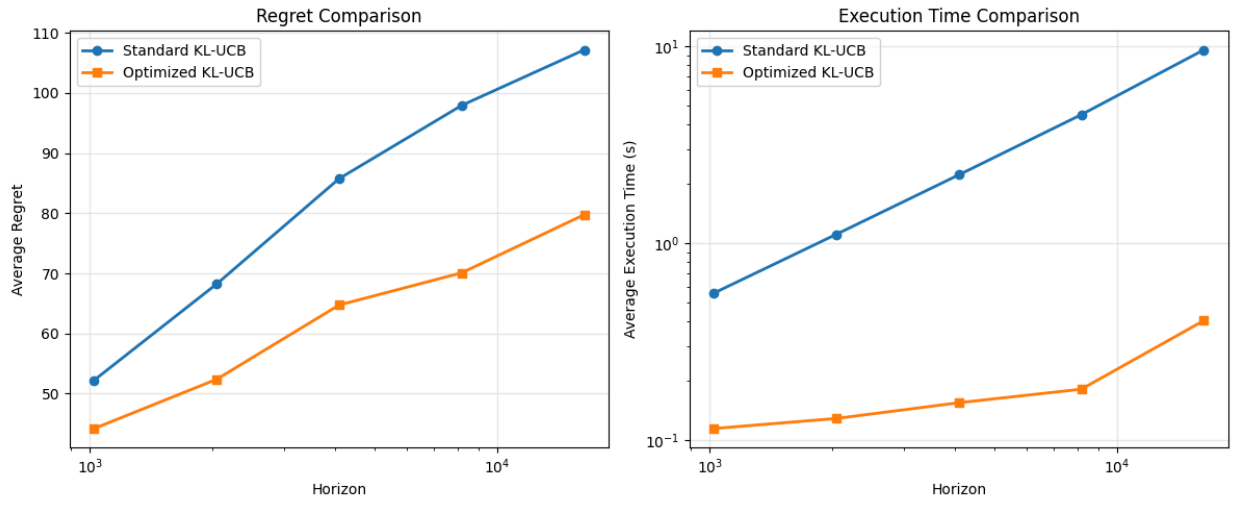
4.1 Regret Optimality

The optimized algorithm maintains the same asymptotic regret bound as standard KL-UCB. This is achieved because the batching strategy only exploits periods where the same arm would be selected repeatedly by the standard algorithm.

4.2 Computational Speedup

- **Best case**: O(T/log T) when long batches are possible
- **Typical case**: O(√T) to O(T/log T) depending on problem structure
- **Worst case**: O(1) when no batching is beneficial

Plots:



Regret and **Time taken comparison for UCB-KL** and **UCB-Optimised**